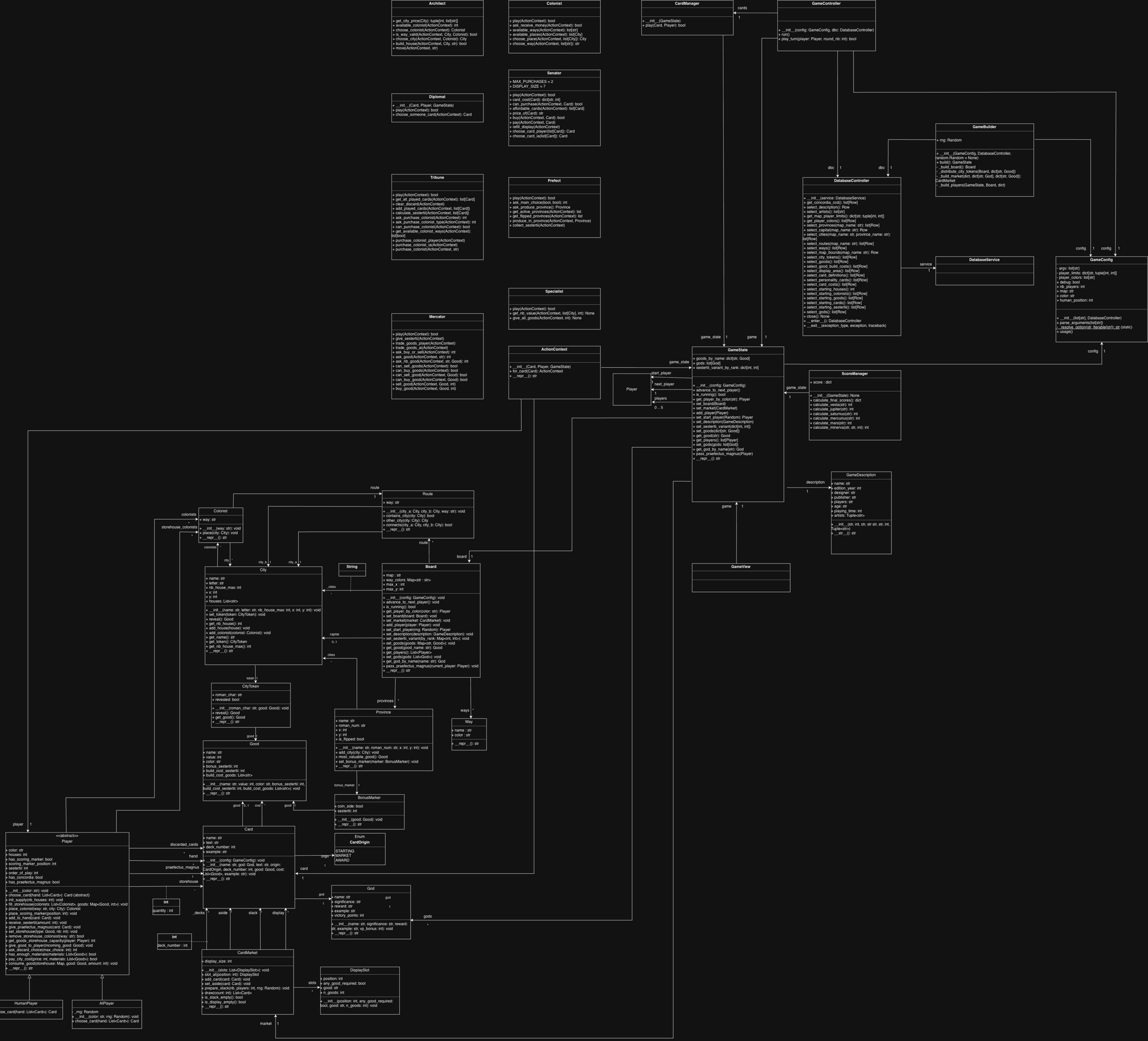

Architect
+ get_city_price(City): tuple[int, list[str]]
+ available_colonist(ActionContext): int
+ choose_colonist(ActionContext): Colonist
+ is_way_valid(ActionContext, City, Colonist): bool
+ choose_city(ActionContext, Colonist): City
+ build_house(ActionContext, City, str): bool
+ move(ActionContext, str)
Colonist
+ play(ActionContext): bool
+ ask_receive_money(ActionContext): bool
+ available_ways(ActionContext): list[str]
+ available_places(ActionContext): list[City]
+ choose_place(ActionContext, list[City]): City
+ choose_way(ActionContext, list[str]): str
CardManager
+ __init__(GameState)
+ play(Card, Player): bool
cards
1
GameController
+ __init__(config: GameConfig, dbc: DatabaseController)
+ run()
+ play_turn(player: Player, round_nb: int): bool
Senator
+ MAX_PURCHASES = 2
+ DISPLAY_SIZE = 7
+ play(ActionContext): bool
+ card_cost(Card): dict[str, int]
+ can_purchase(ActionContext, Card): bool
+ affordable_cards(ActionContext): list[Card]
+ price_of(Card): str
+ buy(ActionContext, Card): bool
+ pay(ActionContext, Card)
+ refill_display(ActionContext)
+ choose_card_player(list[Card]): Card
+ choose_card_ia(list[Card]): Card
Diplomat
+ __init__(Card, Player, GameState)
+ play(ActionContext): bool
+ choose_someone_card(ActionContext): Card
GameBuilder
+ rng: Random
+ __init__(GameConfig, DatabaseController, random.Random = None)
+ build(): GameState
- _build_board(): Board
- _distribute_city_tokens(Board, dict[str, Good])
- _build_market(dict, dict[str, God], dict[str, Good]): CardMarket
- _build_players(GameState, Board, dict)
dbc
1
dbc
1
Tribune
+ play(ActionContext): bool
+ get_all_played_cards(ActionContext): list[Card]
+ clear_discard(ActionContext)
+ add_played_cards(ActionContext, list[Card])
+ calculate_sestertii(ActionContext, list[Card])
+ ask_purchase_colonist(ActionContext): int
+ ask_purchase_colonist_type(ActionContext): int
+ can_purchase_colonist(ActionContext): bool
+ get_available_colonist_ways(ActionContext): list[bool]
+ purchase_colonist_player(ActionContext)
+ purchase_colonist_ia(ActionContext)
+ purchase_colonist(ActionContext, str)
Prefect
+ play(ActionContext): bool
+ ask_main_choice(bool, bool): int
+ ask_produce_province(): Province
+ get_active_provinces(ActionContext): list
+ get_flipped_provinces(ActionContext): list
+ produce_in_province(ActionContext, Province)
+ collect_sestertii(ActionContext)
DatabaseController
+ __init__(service: DatabaseService)
+ get_concordia_ccd(): list[Row]
+ select_description(): Row
+ select_artists(): list[str]
+ get_map_player_limits(): dict[str, tuple[int, int]]
+ get_player_colors(): list[Row]
+ select_provinces(map_name: str): list[Row]
+ select_capital(map_name: str): Row
+ select_cities(map_name: str, province_name: str): list[Row]
+ select_routes(map_name: str): list[Row]
+ select_ways(): list[Row]
+ select_map_bounds(map_name: str): Row
+ select_city_tokens(): list[Row]
+ select_goods(): list[Row]
+ select_good_build_costs(): list[Row]
+ select_display_area(): list[Row]
+ select_card_definitions(): list[Row]
+ select_personality_cards(): list[Row]
+ select_card_costs(): list[Row]
+ select_starting_houses(): int
+ select_starting_colonists(): list[Row]
+ select_starting_goods(): list[Row]
+ select_starting_cards(): list[Row]
+ select_starting_sestertii(): list[Row]
+ select_gods(): list[Row]
+ close(): None
+ __enter__(): DatabaseController
+ __exit__(exception_type, exception, traceback)
service
1
DatabaseService
config
1
config
1
GameConfig
- argv: list[str]
- player_limits: dict[str, tuple[int, int]]
- player_colors: list[str]
+ debug: bool
+ nb_players: int
+ map: str
+ color: str
+ human_position: int
+ __init__(list[str], DatabaseController)
+ parse_arguments(list[str])
- _resolve_option(str, Iterable[str]): str {static}
+ usage()
config
1
Mercator
+ play(ActionContext): bool
+ give_sestertii(ActionContext)
+ trade_goods_player(ActionContext)
+ trade_goods_ai(ActionContext)
+ ask_buy_or_sell(ActionContext): int
+ ask_good(ActionContext, str): int
+ ask_nb_good(ActionContext, str, Good): int
+ can_sell_goods(ActionContext): bool
+ can_buy_goods(ActionContext): bool
+ can_sell_good(ActionContext, Good): bool
+ can_buy_good(ActionContext, Good): bool
+ sell_good(ActionContext, Good, int)
+ buy_good(ActionContext, Good, int)
Specialist
+ play(ActionContext): bool
+ get_nb_value(ActionContext, list[City], int): None
+ give_all_goods(ActionContext, int): None
ActionContext
+ __init__(Card, Player, GameState)
+ for_card(Card): ActionContext
+ __repr__(): str
game_state
1
game
1
GameState
+ goods_by_name: dict[str, Good]
+ gods: list[God]
+ sestertii_variant_by_rank: dict[int, int]
+ __init__(config: GameConfig)
+ advance_to_next_player()
+ is_running(): bool
+ get_player_by_color(str): Player
+ set_board(Board)
+ set_market(CardMarket)
+ add_player(Player)
+ set_start_player(Random): Player
+ set_description(GameDescription)
+ set_sestertii_variant(dict[int, int])
+ set_goods(dict[str, Good])
+ get_good(str): Good
+ get_players(): list[Player]
+ set_gods(gods: list[God])
+ get_god_by_name(str): God
+ pass_praefectus_magnus(Player)
+ __repr__(): str
game_state
start_player
1
next_player
1
players
0 .. 5
Player
game_state
1
ScoreManager
+ score : dict
+ __init__(GameState): None
+ calculate_final_scores(): dict
+ calculate_vesta(str): int
+ calculate_jupiter(str): int
+ calculate_saturnus(str): int
+ calculate_mercurius(str): int
+ calculate_mars(str): int
+ calculate_minerva(str, str, int): int
description
1
GameDescription
+ name: str
+ edition_year: int
+ designer: str
+ publisher: str
+ players: str
+ age: str
+ playing_time: int
+ artists: Tuple<str>
+ __init__(str, int, str, str str, str, int, Tuple<str>)
+ __str__(): str
game
1
GameView
route
1
Route
+ way: str
+ __init__(city_a: City, city_b: City, way: str): void
+ contains_city(city: City): bool
+ other_city(city: City): City
+ connects(city_a: City, city_b: City): bool
+ __repr__(): str
colonists
*
storehouse_colonists
*
Colonist
+ way: str
+ __init__(way: str): void
+ place(city: City): void
+ __repr__(): str
colonists
*
route
*
board
1
city
*
city_b
1
city_a
1
String
_cities
*
Board
+ map : str
+ way_colors: Map<str : str>
+ max_x : int
+ max_y: int
+ __init__(config: GameConfig): void
+ advance_to_next_player(): void
+ is_running(): bool
+ get_player_by_color(color: str): Player
+ set_board(board: Board): void
+ set_market(market: CardMarket): void
+ add_player(player: Player): void
+ set_start_player(rng: Random): Player
+ set_description(description: GameDescription): void
+ set_sestertii_variant(by_rank: Map<int, int>): void
+ set_goods(goods: Map<str, Good>): void
+ get_good(good_name: str): Good
+ get_players(): List<Player>
+ set_gods(gods: List<God>): void
+ get_god_by_name(name: str): God
+ pass_praefectus_magnus(current_player: Player): void
+ __repr__(): str
City
+ name: str
+ letter: str
+ nb_house_max: int
+ x: int
+ y: int
+ houses: List<str>
+ __init__(name: str, letter: str, nb_house_max: int, x: int, y: int): void
+ set_token(token: CityToken): void
+ reveal(): Good
+ get_nb_house(): int
+ add_house(house): void
+ add_colonist(colonist: Colonist): void
+ get_name(): str
+ get_token(): CityToken
+ get_nb_house_max(): int
+ __repr__(): str
capital
0..1
cities
*
token
1
CityToken
+ roman_char: str
+ revealed: bool
+ __init__(roman_char: str, good: Good): void
+ reveal(): Good
+ get_good(): Good
+ __repr__(): str
provinces
*
ways
*
Province
+ name: str
+ roman_num: str
+ x: int
+ y: int
+ is_flipped: bool
+ __init__(name: str, roman_num: str, x: int, y: int): void
+ add_city(city: City): void
+ most_valuable_good(): Good
+ set_bonus_marker(marker: BonusMarker): void
+ __repr__(): str
Way
+ name : str
+ color : str
+ __repr__(): str
good
0..1
Good
+ name: str
+ value: int
+ color: str
+ bonus_sestertii: int
+ build_cost_sestertii: int
+ build_cost_goods: List<str>
+ __init__(name: str, value: int, color: str, bonus_sestertii: int, build_cost_sestertii: int, build_cost_goods: List<str>): void
+ __repr__(): str
bonus_marker
1
BonusMarker
+ coin_side: bool
+ sestertii: int
+ __init__(good: Good): void
+ __repr__(): str
good
0..1
cost
*
good
1
player
1
<<abstract>>
Player
+ color: str
+ houses: int
+ has_scoring_marker: bool
+ scoring_marker_position: int
+ sestertii: int
+ order_of_play: int
+ has_concordia: bool
+ has_praefectus_magnus: bool
+ __init__(color: str): void
+ choose_card(hand: List<Card>): Card {abstract}
+ init_supply(nb_houses: int): void
+ fill_storehouse(colonists: List<Colonist>, goods: Map<Good, int>): void
+ place_colonist(way: str, city: City): Colonist
+ place_scoring_marker(position: int): void
+ add_to_hand(card: Card): void
+ receive_sestertii(amount: int): void
+ give_praefectus_magnus(card: Card): void
+ set_storehouse(type: Good, nb: int): void
+ remove_storehouse_colonist(way: str): bool
+ get_goods_storehouse_capacity(player: Player): int
+ give_good_to_player(incoming_good: Good): void
+ ask_discard_choice(max_choice: int): int
+ has_enough_materials(materials: List<Good>): bool
+ pay_city_cost(price: int, materials: List<Good>): bool
+ consume_good(storehouse: Map, good: Good, amount: int): void
+ __repr__(): str
discarded_cards
*
hand
*
praefectus_magnus
1
storehouse
*
Card
+ name: str
+ text: str
+ deck_number: int
+ example: str
+ __init__(config: GameConfig): void
+ __init__(name: str, god: God, text: str, origin: CardOrigin, deck_number: int, good: Good, cost: List<Good>, example: str): void
+ __repr__(): str
Enum
CardOrigin
STARTING
MARKET
AWARD
origin
1
card
1
int
quantity : int
god
1
God
+ name: str
+ significance: str
+ reward: str
+ example: str
+ victory_points: int
+ __init__(name: str, significance: str, reward: str, example: str, vp_bonus: int): void
+ __repr__(): str
god
1
gods
*
_decks
aside
*
stack
*
display
*
int
deck_number : int
CardMarket
+ display_size: int
+ __init__(slots: List<DisplaySlot>): void
+ slot_at(position: int): DisplaySlot
+ add_card(card: Card): void
+ set_aside(card: Card): void
+ prepare_stack(nb_players: int, rng: Random): void
+ draw(count: int): List<Card>
+ is_stack_empty(): bool
+ is_display_empty(): bool
+ __repr__(): str
slots
*
DisplaySlot
+ position: int
+ any_good_required: bool
+ good: str
+ n_goods: int
+ __init__(position: int, any_good_required: bool, good: str, n_goods: int): void
market
1
HumanPlayer
+ choose_card(hand: List<Card>): Card
AIPlayer
- _rng: Random
+ __init__(color: str, rng: Random): void
+ choose_card(hand: List<Card>): Card